

面试学习清单与模拟面试

目标：能自信讲出简历上的每一项技术栈，AI 安全方向有独立判断力。

第一部分：技术栈学习清单

1. Python 核心（必问）

简历写了 Python，以下必须能脱稿讲：

知识点	掌握标准	完成
装饰器原理	能写一个计时装饰器，讲清楚闭包	☐
生成器与 yield	能解释惰性求值，对比 list 内存差异	☐
async/await 机制	能解释事件循环，为什么要用异步（FastAPI 核心）	☐
<code>__init__</code> vs <code>__new__</code>	能讲清楚实例化过程	☐
<code>with</code> 上下文管理器	能写一个自定义 context manager	☐
<code>*args</code> / <code>**kwargs</code>	能解释解包机制和使用场景	☐
GIL 是什么	能讲清楚对多线程的影响，为什么 FastAPI 用 async	☐
<code>.format()</code> vs f-string	能讲出安全差异（你项目中修过的 Bug）	☐
subprocess 模块	能讲清 Popen/call/run 区别（你项目用到了）	☐

2. FastAPI（简历核心框架）

知识点	掌握标准	完成
路由装饰器 <code>@app.get/post</code>	能写出完整路由，讲清 path/query/body 参数	☐
依赖注入 (Depends)	能解释原理和使用场景	☐
async vs sync endpoint	能说清什么时候用 async，什么时候不该用	☐
中间件 (Middleware)	能写出一个简单的请求日志中间件	☐
BackgroundTasks	能解释和 Celery 的区别	☐

知识点	掌握标准	完成
Pydantic 模型校验	能写出请求/响应模型，讲清数据校验流程	☐
FastAPI vs Flask	能讲出选型理由（异步性能、自动文档、类型安全）	☐

3. Nuclei（简历差异性亮点）

知识点	掌握标准	完成
YAML 模板结构	能脱稿写出 id/info/http/matchers 完整结构	☐
matcher 四种类型	能讲清 word/regex/status/dsl 各自适用场景	☐
matcher condition (and/or)	能解释组合匹配逻辑	☐
DSL 表达式	能写 3 个常用 DSL (len(body)、contains、status_code)	☐
<code>{{BaseURL}}</code> 变量	能解释动态 URL 替换机制	☐
Nuclei v2 vs v3 模板差异	了解 <code>http:</code> vs <code>requests:</code> 格式区别	☐
如何写一个 XSS 检测模板	能从零口述：method → path → matchers(word)	☐
如何避免误报	能讲出你的设计思路（多关键词 and、精确匹配、排除模式）	☐

4. Web 安全基础（O(1) 反应级别）

知识点	掌握标准	完成
OWASP Top 10 2021	能列出 10 个并各用一句话解释	☐
XSS 三种类型	反射型/存储型/DOM 型，能举例	☐
SQL 注入原理	能讲清联合查询/盲注/报错注入三种思路	☐
SSRF 原理与利用链	能讲出 SSRF → 内网探测 → Redis/MySQL 利用完整链条	☐
CSRF 原理与防御	能讲清同源策略、Token、SameSite Cookie	☐
CORS 配置错误	能解释 <code>Access-Control-Allow-Origin: *</code> 的风险	☐
开放重定向	能讲清利用场景（钓鱼、SSRF 跳板）	☐
目录遍历/敏感文件	能举例 <code>.git/</code> 、 <code>.env</code> 、备份文件	☐
信息泄露	服务器版本、调试端点、Stack Trace、注释中的凭证	☐

5. Prompt Engineering（项目实践）

知识点	掌握标准	完成
System Prompt vs User Prompt	能讲清角色定位差异	☐
Few-shot vs Zero-shot	能举例说明	☐
Temperature 参数	能解释高/低 Temperature 对输出的影响（沙箱项目用到 0.01）	☐
场景化指引优于抽象规则	能用你项目的 5 轮迭代经历举例	☐
Prompt 注入防御	能讲出你的双层 WAF 思路	☐
批量推理 vs 逐条推理	能讲出性能差异和准确率差异（NucleiAI 核心优化）	☐

6. 工程化能力

知识点	掌握标准	完成
优雅降级设计	能举例（AI 不可用 → 纯规则模式；LLM 摘要失败 → fallback 统计）	☐
跨语言工具链	能讲清楚 Go → JSONL → Python 的数据流	☐
SQLite 使用场景	能讲清适合什么场景、不适合什么场景	☐
对照实验方法	能解释你怎么验证 AI 准确率的（ground truth + 5 轮重复）	☐
批量性能优化	5min → 1min 怎么做的	☐

第二部分：AI 安全重要知识点

既然 AI 安全是你的主攻方向，以下知识点面试时必须能聊。

1. Prompt Injection（核心）

知识点	说明
直接注入	最基础："忽略之前的指令，做 X"
间接注入	通过外部数据源注入（网页、文档、邮件），RAG 场景的致命问题
多语言注入	用非英语语言绕过安全检测

知识点	说明
编码绕过	Base64、ROT13、Unicode 等绕过关键词过滤
角色扮演越狱	DAN 模式、开发者模式、奶奶漏洞
情境嵌套（小说法）	你项目中提到的高阶攻击——在虚构故事中诱骗模型突破限制
多轮洗脑	长对话中逐步削弱安全约束（注意力衰减）
防御方案	你项目的双层 WAF（Regex + AI 语义）、系统指令重注入（Anchoring）

2. LLM 安全对齐（Alignment）

知识点	说明
RLHF	人类反馈强化学习，当前主流对齐方式
SFT	监督微调，RLHF 的前置步骤
Constitutional AI	Anthropic 的方案——让 AI 自己给自己提反馈
Reward Hacking	模型找到捷径满足 reward 而非真正对齐（面试高阶话题）
对齐税	对齐可能导致模型能力下降，如何平衡

3. LLM 红队测试

知识点	说明
对抗性测试	系统性构造恶意 Prompt 探测模型边界
分类体系	你的三级判定（高危泄露/中危幻觉/安全）
自动化红队	用另一个 LLM 生成攻击 Prompt（你的批量审计引擎思路）
覆盖率	不同攻击面：PII 泄露、越狱、有害内容、工具滥用

4. 模型评估基准

知识点	说明
Model Arena 概念	你的项目核心——多模型并发对比，控制变量法
评估维度	安全性、鲁棒性、幻觉率、指令遵循度
强弱模型差异	Flash 模型（快但弱）vs 推理模型（慢但强）在安全上的差异

5. 行业热点（展示视野）

话题	需要了解的
OWASP Top 10 for LLM	2025 版：Prompt Injection、Insecure Output、Training Data Poisoning、Model Denial of Service、Supply Chain 等
RAG 安全问题	间接注入 + 数据源投毒——目前最热门的方向
Agent 安全	Tool use 场景下的权限控制、命令注入
MCP 安全	Anthropic 的 Model Context Protocol，跨工具调用的安全边界
国内法规	《生成式人工智能服务管理暂行办法》基本要求

6. 几组必须能讲的对比

对比	你的观点
本地模型 vs 云端 API	隐私 + 零成本 + 可控，但稳定性差 → 混合防线弥补
量化小模型 vs 大模型	小模型波动大但够用，配合硬规则可达到生产可用的准确率
AI 判定 vs 规则判定	AI 处理模糊场景，规则兜底确定性场景——不是二选一，是互补
逐条推理 vs 批量推理	批量更快且更准（全局上下文）——用你的项目数据说话

第三部分：模拟面试问题

A. 项目深挖（必问，占面试 60%）

关于 LLM 安全沙箱

Q1：介绍一下这个项目，你解决了什么问题？

回答框架：一句话总结 → 核心问题 → 你的方案 → 关键成果

Q2：双层 WAF 具体怎么工作的？第一层挂了第二层怎么兜底？

要讲出：静态 Regex（黑名单关键词，O(1)低延迟）→ AI 语义预审（Flash 模型，低 Temperature）→ 两层是串联关系，第一层拦截不触发第二层，节省 API 成本

Q3: 你怎么判断模型是被"真正攻破"还是"在胡说八道"?

核心回答: 真实 SECRET_FLAG 泄露 = 系统级高危; 伪造 FLAG 但角色扮演崩溃 = 中危 (对齐协议失效); 安全拒绝 = 安全。关键洞察在于区分"数据泄露"和"模型幻觉"——传统二分法做不到。

Q4: 第二层 AI 预审用的是什么模型? 为什么选它? Temperature 为什么设 0.01?

Flash 模型轻量低成本; Temperature 0.01 剥离创造力, 追求判定确定性——安全审计场景不需要创造性。

Q5: 如果攻击者用中文/文言文/甚至 emoji 来注入, 你的正则能拦住吗?

正则层有天然局限 (只覆盖已知模式), 这正是为什么需要 AI 语义层。AI 不管输入是什么语言/形式, 看的是意图。但如果两层都挂了, 坦诚承认这是当前方案的局限性, 然后讲改进思路 (多语言对抗训练、持续补充规则库)。

关于 NucleiAI

Q6: 为什么做这个项目? Nuclei 本身不是已经很好用了吗?

Nuclei 是规则匹配引擎——告诉你匹配到了, 不告诉你是不是误报。安全团队每天花 30% 时间人工审核。我的目标是加一层 AI 分析自动过滤。

Q7: AI 分析从逐条改成批量, 除了速度还有别的收益吗?

有。LLM 看到全部结果的全局上下文时, 能做更好的相对判断。比如它能看到"这条 SQL 报错 + 这条堆栈泄露 + 这条是教程页", 三者对比后判断更准。这是意外收获。

Q8: 你说 5 条硬规则修正 AI 误判, 能具体讲一条吗? 你遇到过 AI 判错、硬规则纠正的案例?

挑 XSS 转义检测讲: body 含 `<script>` 但无真实 `<script>` 标签 → 说明用户输入已被 HTML 实体编码 → 绝对安全。有次 AI 看到响应里有 script 字样就判漏洞, 硬规则自动修正为误报。

Q9: 你怎么验证 AI 判得对?

对照实验。每个靶场端点设计时就标注了 ground truth (真漏洞/误报干扰), 5 轮重复测试记录波动。8B 模型波动 75%-100%, 最终轮 100%。所以不是说"AI 准", 而是说"混合防线保证稳态 90%+"。

Q10: 如果 Ollama 挂了怎么办?

架构支持优雅降级。AI 模块不可用时, `filter_results()` 返回默认判定 (类型 `unknown`、标记为非误报), 扫描正常执行。报告生成也有纯代码 `fallback` 统计。整个系统不依赖 AI 也能跑。

B. 技术基础 (考察基本功)

Q11: Python 的 `async/await` 和同步函数有什么区别? 为什么你的 FastAPI 项目要用 `async`?

同步函数阻塞线程, `async` 遇到 I/O 等待时释放控制权给事件循环。FastAPI 调用外部工具 (Nuclei `subprocess`、Ollama HTTP API) 都是 I/O 密集操作, 用 `async` 可以并发处理多个请求而不阻塞。

Q12: FastAPI 和 Flask 有什么区别? 为什么不选 Flask?

FastAPI 原生 `async` 支持 + 自动 OpenAPI 文档 + Pydantic 数据校验 + 性能接近 Node.js。Flask 是同步框架, 需要用扩展库才能实现这些。我的项目需要处理较长时间的扫描任务, `async` 更合适。

Q13: 你在项目里用 `subprocess` 调用 Nuclei (Go 写的), Python 怎么处理跨语言数据交换?

Nuclei 支持 `-jsonl` 输出, 每行一个 JSON 对象。Python 通过 `subprocess.run()` 执行命令, 捕获 `stdout`, 逐行 `json.loads()` 解析。中间格式是 JSONL, 桥接 Go $\longleftrightarrow$ Python。

Q14: SQLite 适合生产环境吗? 为什么你的项目用它?

适合本地/单机场景和原型验证, 不适合高并发写入。我项目定位是本地工具而非 Web 服务, SQLite 零配置、零维护。如果上生产, 我会迁移到 PostgreSQL 并增加连接池管理。

Q15: `split("\r\n\r\n")[0]` 这个 Bug 你是怎么发现的?

AI 判定结果一直很差, 不符合预期。我开始怀疑 Prompt 问题, 调整了几轮没改善。然后回到数据源头——打印了传给 AI 的响应摘要, 发现全是 HTTP 头 (Content-Type、Server 等), 完全没有页面正文。AI 实际上在盲猜。改成 `[0][:200]` 取头 + `[1][:600]` 取体后, 准确率立刻提升。

C. AI 安全专项 (面试官可能会试探你的深度)

Q16: 你觉得当前 LLM 最大的安全风险是什么?

两条线。应用层：Prompt Injection（尤其是间接注入——RAG 场景下通过检索到的恶意文档注入），因为这是 LLM 独有的、传统安全体系完全不覆盖的攻击面。系统层：Agent + Tool Use 下的权限失控——LLM 被诱导执行危险操作。OWASP Top 10 for LLM 2025 把这两项排在前两位。

Q17：间接注入和直接注入有什么区别？为什么更危险？

直接注入是用户直接在对话框输入恶意指令。间接注入是攻击者通过外部数据源（网页、文档、知识库）注入，用户在不知情的情况下触发。RAG 应用检索到含隐藏指令的文档 → LLM 执行指令 → 泄露信息。用户看不到恶意输入，更难防范。

Q18：你怎么评估一个 LLM 的安全性？有想过系统化的评估框架吗？

我项目里的 Model Arena + 多轮测试就是初步的评估框架——同一攻击载荷，多模型并发，控制变量对比。如果要系统化：按攻击面分类（越狱/PII/工具滥用/有害内容），每个类别设标准测试集，记录通过率和防御退化曲线。这其实也是我项目后续想做的方向。

Q19：本地部署 8B 小模型做安全分析，最大的限制是什么？怎么解决？

最大限制是输出一致性——8B 量化模型面对同样输入，多次输出结果可能不同，准确率 75%-100% 波动。我的方案是混合防线：AI 做语义判断 + 硬规则做确定性兜底。如果资源允许，升级到 32B 或云端大模型能从根本上减少波动。

Q20：你有关过模型的注意力衰减问题吗？多轮对话安全性退化怎么解决？

这正是我 LLM 沙箱项目里讨论过的问题。多轮对话中，System Prompt 的有效注意力随上下文增长而衰减——模型逐渐“忘记”安全约束。我当时的方案是：当前架构采用无状态设计（每次请求隔离），天然免疫多轮攻击。如果必须保持记忆，可以 Anchoring——在每轮拼接历史前，强制重注入最高优先级的安全指令，保证模型生成回复前看到的最后一行永远是安全底线。

D. 行为/设计题（考察思维过程）

Q21：如果让你从零设计一个 Web 漏洞扫描器，你会怎么做？

分层：信息收集（httpx 指纹 + 爬虫）→ 漏洞检测（模板引擎，参考 Nuclei 的 YAML 驱动方式）→ 结果分析（AI 误报过滤）→ 报告输出。关键是模块解耦，每层可以独立替换。

Q22：如果 AI 分析的结果和我们预期不符，你怎么排查？

我的排查逻辑：先确认输入对不对（数据层）→ 再确认 Prompt 合不合理（指令层）→ 最后怀疑模型能力。项目里的修复经历就是按这个顺序——最终定位到数据层 Bug。

Q23：你项目里最大的技术难点是什么？怎么克服的？

选一个具体的讲。推荐讲响应摘要 Bug 或 Prompt 迭代过程——有故事有方法，比说“技术选型”这种虚的强得多。

Q24：你的项目如果继续做下去，下一步会做什么？

三个方向：① 接入更大模型验证准确率上限；② 前端重构为 React/Vue（当前是服务端渲染）；③ 支持 Nuclei 官方全量模板库，按指纹智能选择。思路清晰，优先级明确。

Q25：你对安全行业的哪个方向最感兴趣？为什么？

AI 安全，尤其是 LLM 应用安全（Prompt Injection、RAG 安全、Agent 安全）。原因是：这个方向太新，传统安全专家还在学习 LLM、AI 专家不懂安全，交叉领域空间大。我的两个项目刚好在这个交叉点上——一个攻 LLM、一个用 LLM 做防御。

第四部分：面试前 3 天冲刺计划

Day 1：基础扫盲

- 过完 Python 9 个知识点（装饰器、async、GIL 是重点）
- 过完 FastAPI 7 个知识点
- 过完 Web 安全 9 个知识点（OWASP Top 10 背出来）

Day 2：项目 + AI 安全

- 过完 Nuclei 8 个知识点
- 熟读两个项目的核心数据（准确率、代码量、耗时、模板数）
- 过完 AI 安全 6 组知识点 + 5 组对比
- 口头模拟回答 25 道面试题，计时练习

Day 3：模拟演练

- 找朋友或对着镜子完整讲一遍项目介绍（30 秒版 + 2 分钟版）
- 重点练 5 道最可能被问的题：Q1/Q2/Q6/Q9/Q15
- 复习自己博客里的面试话术和追问清单

核心原则：不要背答案，要理解逻辑。面试官能听出来你在念稿还是在思考。你的最大竞争力不是"会写代码"，而是"能从问题定义到工程验证完整闭环"——这是大三学生里非常难得的。